Lab 11

Graphical User Interface using JavaFX (Part 1)

Objectives: ***At the end of this lab session the student should be able to:***

- Create interactive applications using JavaFX.
- Handle events generated by user interactions with GUIs.

Prerequisites:

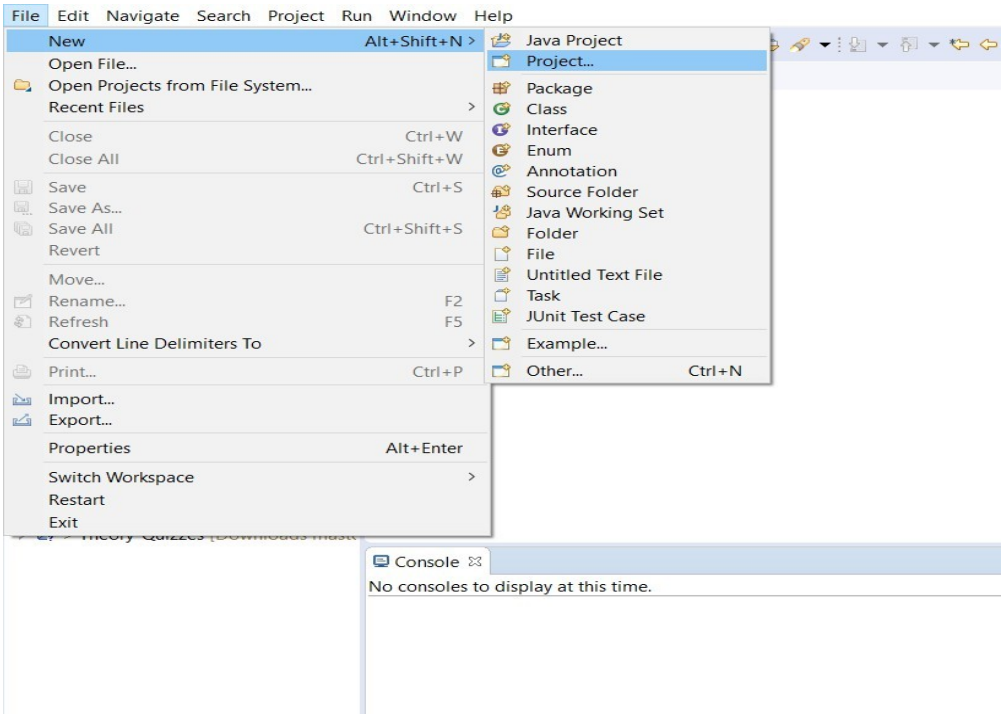
Download the Software needed for JavaFX (Refer to the uploaded Installation Guide)

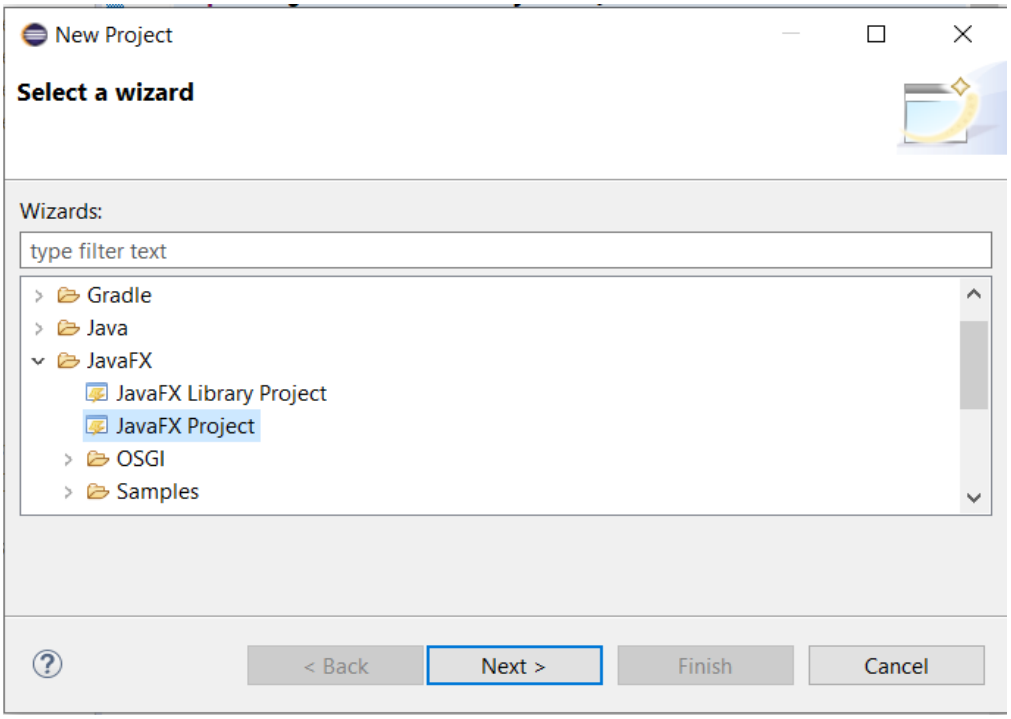
Introduction:

1. GUI provides a visually appealing and user-friendly interface for developing interactive applications.
2. JavaFX is a powerful tool for creating GUI-based applications.
3. The key elements of a JavaFX application are:
    - **Stage:** It is the main window of JavaFX application.
    - **Scene:** It is the container that holds all the visual elements (UI Components) within your application. Each Stage can have one or more Scene objects, allowing you to switch between different views.
    - **UI Components:** They are the building blocks for your GUI, also known as controls. These include buttons, labels, text fields, checkboxes, menus, and more.
    - **Node:** In the context of JavaFX, a Node is anything that can be displayed within a Scene. The root node, typically the first Scene hierarchy, serves as the starting point and has no parent.
    - **Layout Containers:** They are used to help you organize and manage the positioning of UI components within a parent Node. Common layout containers include VBox, HBox, GridPane, StackPane, and BorderPane, each offering different layout strategies to suit your application's needs.
    - **Event Handling:** It enables your application to respond to user actions, such as button clicks, mouse movements, and keyboard input. With event handling, you can specify what should happen when each user interaction occues, allowing you to create dynamic and interactive applications.

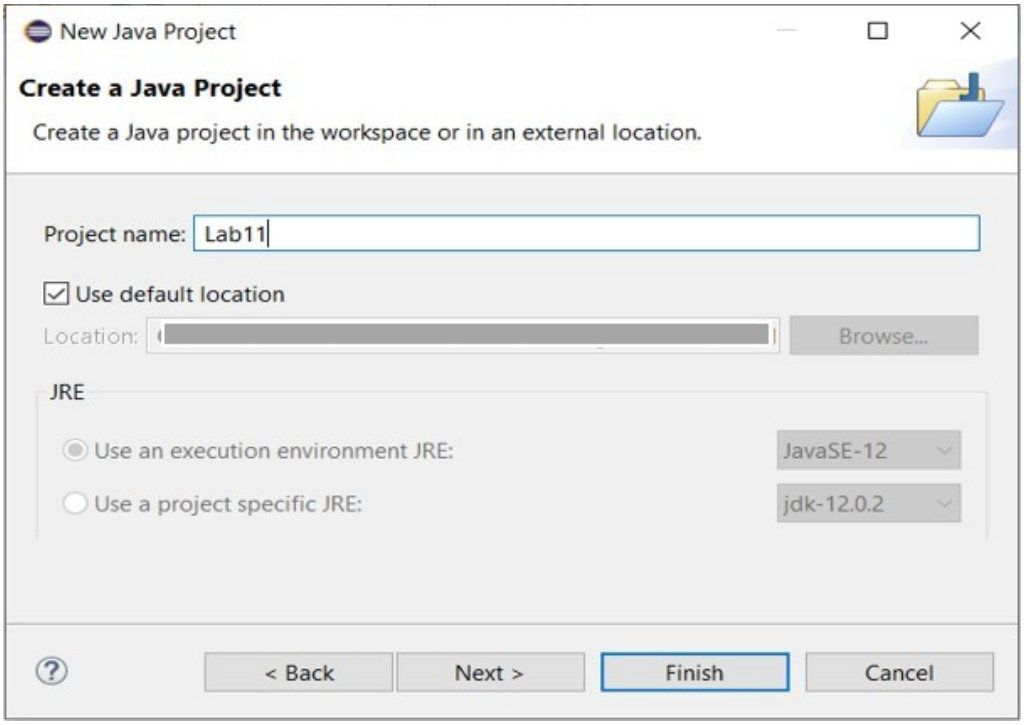
Part 1: Creating JavaFX project:

Step 1: Go to Files → New → Project → JavaFX Project → Next.

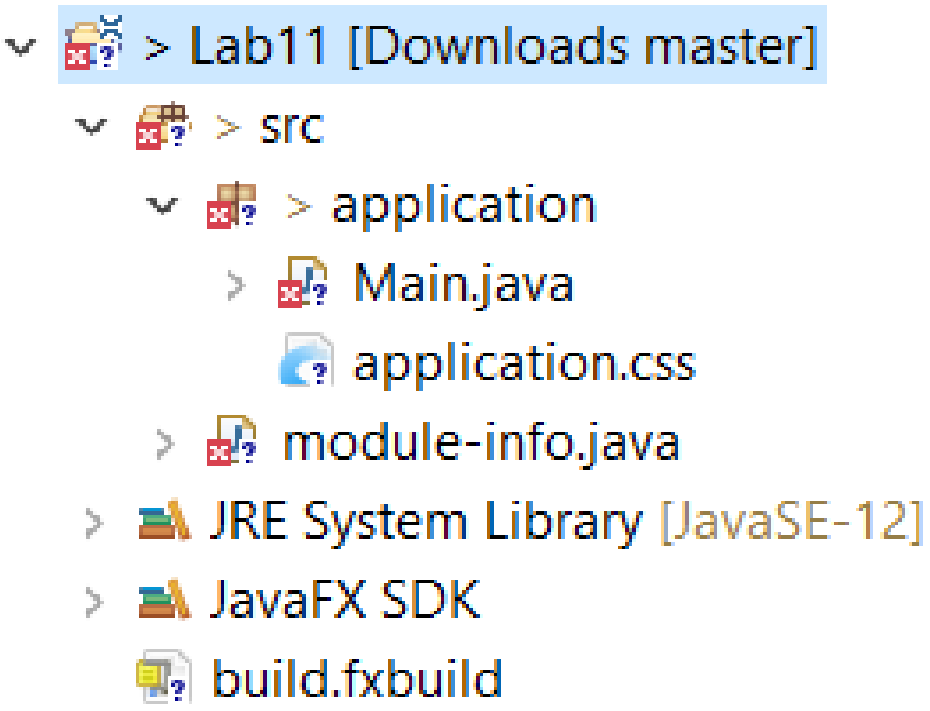




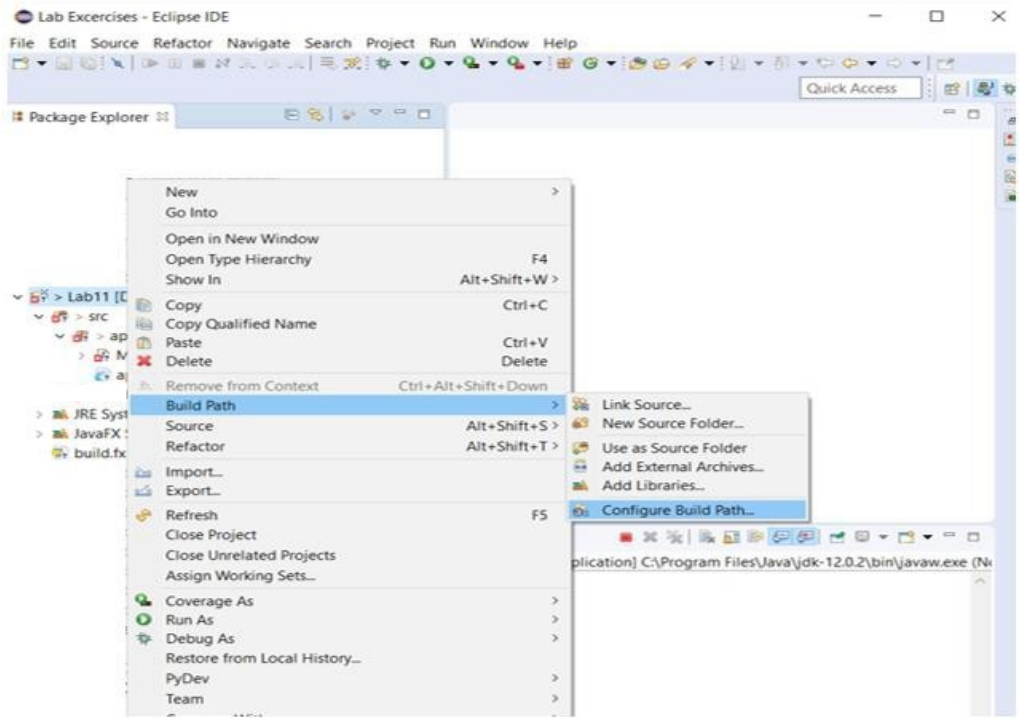
Step 2: Name your project as Lab11, and then click on Finish to proceed.

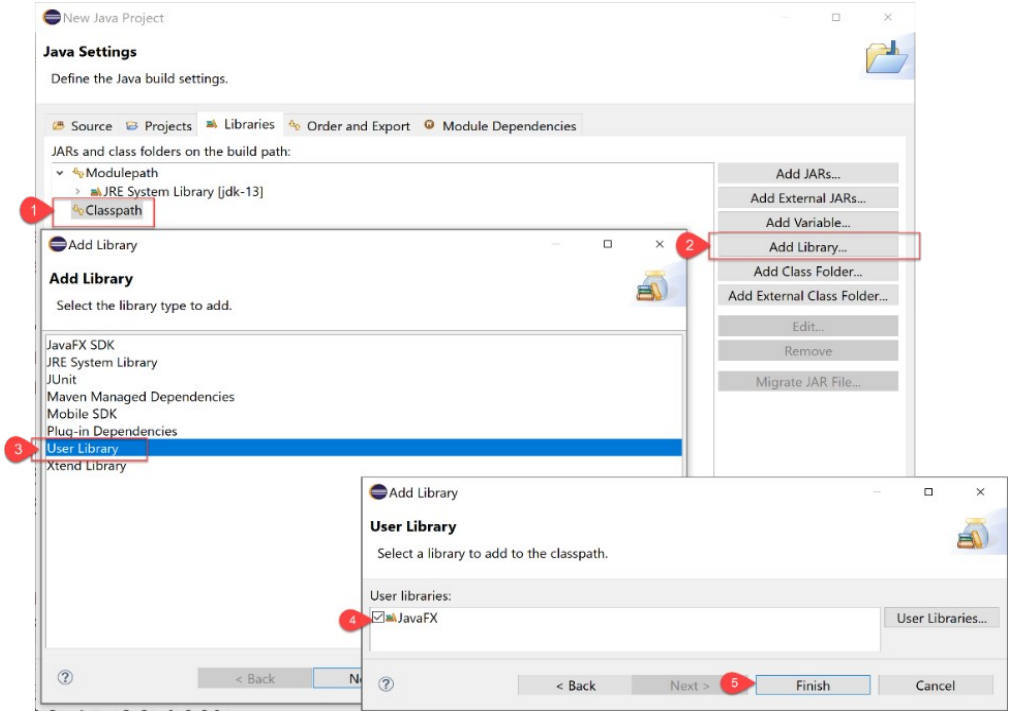


Step 3: Delete the module-info.java file.



Step 4: Configure the path to the JavaFX libraries file. This is done by right-clicking on the project → Build Path → Configure Build Path → Classpath → Add Library → User Library → Check the JavaFX box → Finish.

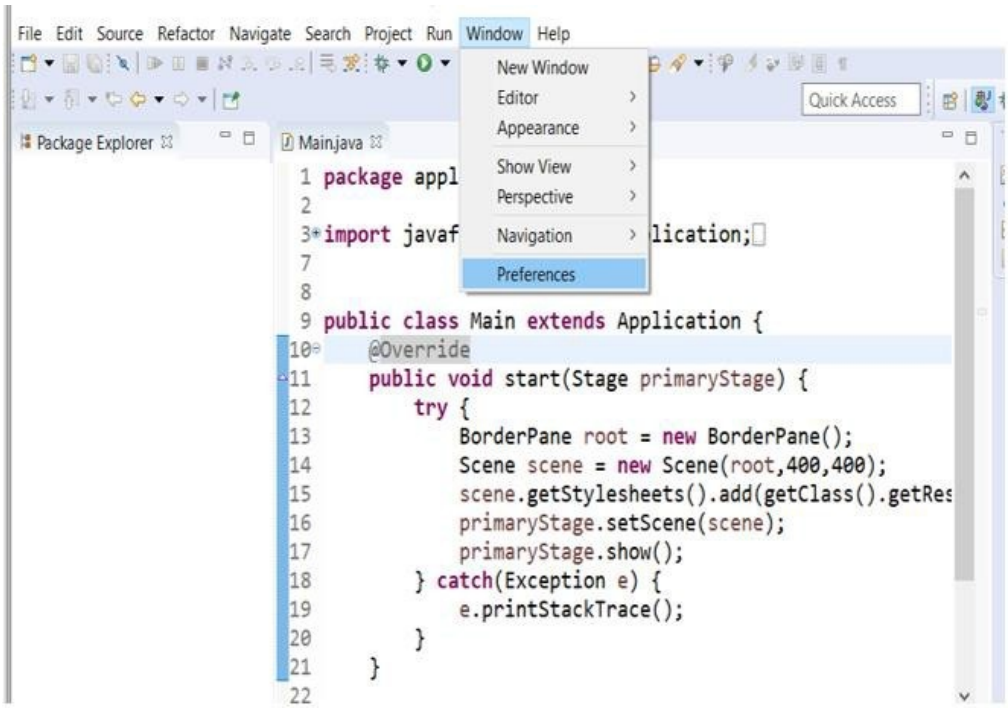


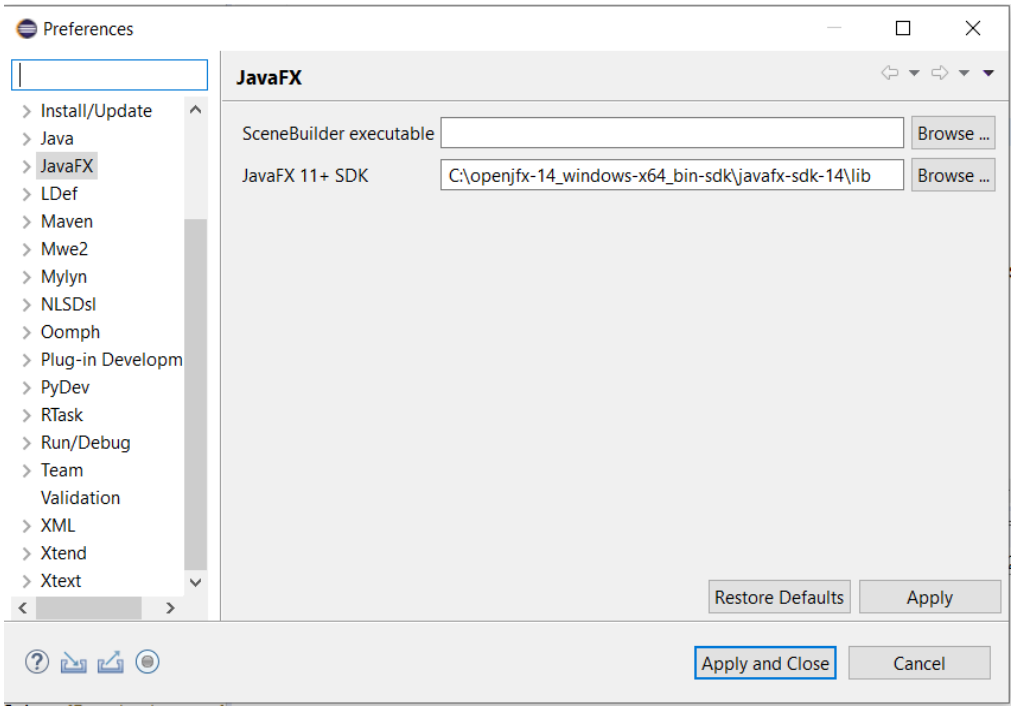


Step 5: Run your program.

If you encountered the following error message, use these steps to solve it: Go to Window → Preferences → JavaFX → Provide the path of the JavaFX SDK libraries to be used → Apply and Close

> Error occurred during initialization of boot layer java.lang.module.FindException: Module javafx.controls not found



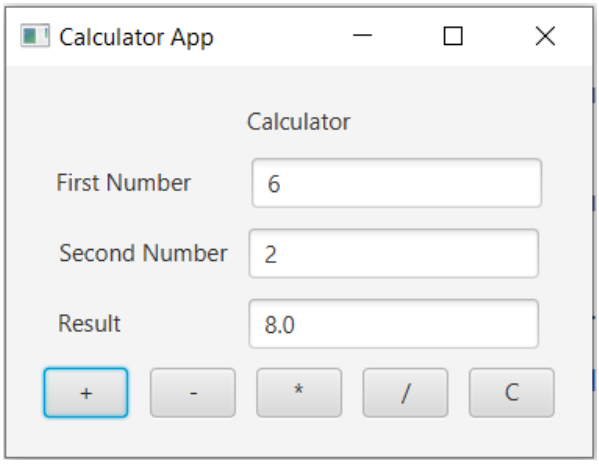


Part 2: Problem Solving Exercises

Exercise 1: Calculator Application

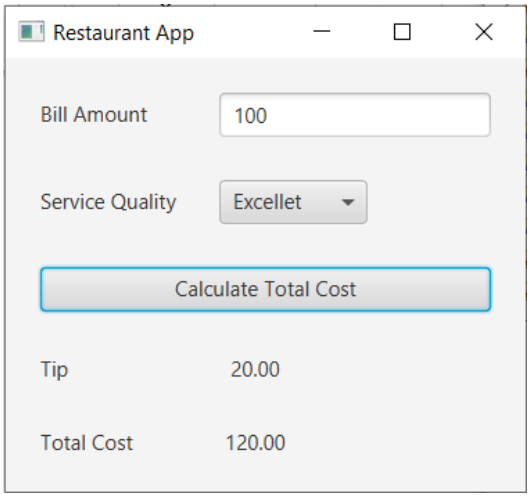
- Use the project you just created to solve this exercise.
- Implement the following to achieve a simple Calculator application.

1. Choose the BorderPane as the root layout.
2. Resize the Scene to be 300 pixels in width and 200 pixels in height.
3. Set the window title (Stage) as ‘Calculator App’.
4. Use the needed UI components (labels, text fields, buttons).
5. Make sure that the result text field is **not editable**.
6. Use ‘50’ as the width of the created buttons (use the setPrefWidth(value) method).
7. Set the event handler for each button to perform the correct operation and display the result in the result text field. The ‘C’ button, when clicked, clears all the text fields.
8. Add some padding for a cleaner look.



Exercise 2: Restaurant Bill Application

- Implement the following to achieve a simple Restaurant Bill application.



1. Create a new Java project named "Lab11-Ex2" and configure it to use JavaFX.
2. Choose the GridPane as the root layout.
3. Resize the Scene to be 300 pixels in width and 250 pixels in height.
4. Set the window title (Stage) as 'Restaurant App'.
5. We would like to add a tip based on three perceived levels of services. This is a good chance to use an Enum. Create a new Enum called ServiceQuality for this task. Use the following code:

```
public enum ServiceQuality {
        Excellet(.2), //20% Good(.15),
        //15% Poor(.1); //10%

        private double tip;
        private ServiceQuality(double tip) {
                this.tip = tip;
        }

        public double getTip() {
                return tip;
        }
    }
```

6. Use the needed UI components (labels, text fields, combo box, buttons). To create a combo box for the service quality label, use the following code:

```
ComboBox<ServiceQuality> serviceQualityCBox = new ComboBox<ServiceQuality>();
serviceQualityCBox.getItems().addAll(ServiceQuality.values());
```

7. Make sure that the 'Calculate Total Cost' button spans all the columns when added to the Scene.
8. Add an event handler for the button to respond when clicked. Use the following code for the handler.

```
calcBtn.setOnAction(new EventHandler<ActionEvent>(){
    public void handle(ActionEvent event) {
            double billAmount=Double.parseDouble(costTextField.getText()); ServiceQuality
            serviceQuality =
            serviceQualityCBox .getSelectionModel().getSelectedItem(); double tip = billAmount
            * serviceQuality.getTip(); root.addColumn(1, new Label(String.format("%7.2f", tip)));
            root.addColumn(1, new Label(String.format("%7.2f", billAmount
            +tip)));
            }
    });
```

9. Add some padding for a cleaner look. Set both the vertical and horizontal gaps of the grid layout to 25 pixels.

Part 3: Additional Practice Exercise

Build on what you did in Exercise 1 to implement a more sophisticated calculator application as follows:

